APACHE KAFKA TRAINING PROGRAM

DAY 3

Monitoring, Troubleshooting & Diagnostics

Study Material

Level: Intermediate-Advanced

Day 3 Overview

Day 3 builds on the foundational and administrative knowledge from Days 1 & 2. Today you will master how to monitor Kafka clusters for health and performance, diagnose real-world issues efficiently, and apply structured troubleshooting methodology used by Kafka L3 support engineers.

Learning Objectives

- Understand and interpret all critical Kafka JMX and operational metrics
- Identify, diagnose, and resolve consumer lag issues
- Troubleshoot Under-Replicated Partitions (URPs) and broker failures
- Analyze Kafka broker and controller logs effectively
- Simulate and resolve common production incidents
- Apply structured diagnostic workflows for rapid incident resolution

Module 1: Key Kafka Metrics

Effective Kafka operations begin with understanding which metrics matter, what they indicate, and what thresholds trigger concern. Kafka exposes metrics via JMX (Java Management Extensions) and most production deployments integrate these with Prometheus and Grafana.

1.1 Broker Health Metrics

1.1.1 Active Controller Count

The kafka.controller:type=KafkaController,name=ActiveControllerCount metric must always equal 1 across the entire cluster. This metric tells you which broker is acting as the active controller — the brain of your Kafka cluster.

- **Value = 1 on exactly one broker: Healthy**
- Value = 0 on all brokers: No active controller — cluster is in a crisis state
- Value > 1 on any broker: Split-brain — two brokers think they are the controller

> **Critical Rule:** If ActiveControllerCount is 0 or > 1, your cluster cannot perform partition leadership elections. Producers may stall and consumers may stop reading until this is resolved.

1.1.2 Under-Replicated Partitions (URP)

Under-Replicated Partitions (URP) is one of the most important metrics in Kafka. It indicates the number of partitions where one or more replicas are not in sync with the leader.

> *MBean: kafka.server:type=ReplicaManager,name=UnderReplicatedPartitions*

- **Value = 0: All replicas are in sync. Healthy state.**
- Value > 0: Some partitions are under-replicated. Investigate immediately.
- Sustained high URP: Indicates a failing broker, network issue, or disk slowdown

Common causes of URPs:

- A broker is down or being restarted
- A follower is too slow to keep up with the leader (controlled by replica.lag.time.max.ms)
- Network congestion between brokers
- Disk I/O bottleneck on a follower broker

1.1.3 Offline Partitions Count

Represents the number of partitions that have no active leader. An offline partition means producers cannot write and consumers cannot read from it.

> *MBean: kafka.controller:type=KafkaController,name=OfflinePartitionsCount*

> **Alert Threshold:** Any value > 0 for OfflinePartitionsCount should trigger an immediate PagerDuty/alert. This is a production-impacting event.

1.1.4 Request Handler Idle Ratio

Measures what fraction of time the I/O threads are idle. A healthy broker spends significant time idle, waiting for work.

Idle Ratio	Interpretation	Action
> 0.5 (50%)	Healthy — broker has capacity headroom	No action needed
0.2 – 0.5 (20-50%)	Moderate load — monitor closely	Review producer/consumer rates
< 0.2 (< 20%)	Broker is overloaded — immediate action needed	Scale out or reduce load

1.1.5 Network Request Metrics

These metrics track the latency and throughput of network requests to the broker:

- kafka.network:type=RequestMetrics,name=TotalTimeMs — Total time to process a request
- kafka.network:type=RequestMetrics,name=RequestQueueTimeMs — Time a request spends queued
- kafka.server:type=BrokerTopicMetrics,name=BytesInPerSec — Incoming bytes per second
- kafka.server:type=BrokerTopicMetrics,name=BytesOutPerSec — Outgoing bytes per second
- kafka.server:type=BrokerTopicMetrics,name=MessagesInPerSec — Message ingestion rate

1.1.6 Log Flush Metrics

Kafka buffers writes in memory (page cache) before flushing to disk. Monitoring flush latency is critical for latency-sensitive applications:

- kafka.log:type=LogFlushStats,name=LogFlushRateAndTimeMs — Flush rate and duration

> **Note:** High log flush latency (> 1000ms) often indicates disk I/O problems. Kafka's default is to rely on OS page cache flushing, but this can be tuned via log.flush.interval.messages and log.flush.interval.ms.

1.2 Consumer Lag Metrics

Consumer lag is the delta between the latest offset produced to a partition and the latest offset committed by a consumer group. It is the most operationally significant metric for end-to-end pipeline health.

1.2.1 Understanding Consumer Lag

- Log End Offset (LEO): The offset of the next message to be written to a partition
- Current Offset: The last offset committed by the consumer group
- Consumer Lag = Log End Offset - Current Offset

> *Example: If a partition has LEO = 1,000,000 and the consumer group has committed offset 900,000, the lag for that partition is 100,000 messages.*

1.2.2 Checking Consumer Lag via CLI

The kafka-consumer-groups.sh tool is your primary CLI tool for checking lag:

```
# List all consumer groups
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --list
```

```
# Describe a specific consumer group
kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --describe --group my-consumer-group
```

```
# Output columns: GROUP, TOPIC, PARTITION, CURRENT-OFFSET, LOG-END-OFFSET, LAG,
CONSUMER-ID, HOST, CLIENT-ID
```

Interpreting the output:

Column	Description	Concern If...
CURRENT-OFFSET	Last offset committed by this consumer	Stops increasing while LEO keeps growing
LOG-END-OFFSET	Latest offset in the partition	Growing much faster than CURRENT-OFFSET
LAG	LEO - Current Offset	Steadily increasing or very large
CONSUMER-ID	Active consumer instance	Shows '-' (no active consumer)

1.2.3 Consumer Lag MBean Metrics

For programmatic monitoring, use the following JMX metrics on the consumer side:

- kafka.consumer:type=consumer-fetch-manager-metrics,client-id=X,name=records-lag-max — Max lag across all assigned partitions
- kafka.consumer:type=consumer-fetch-manager-metrics,client-id=X,name=records-lag — Lag per partition
- kafka.consumer:type=consumer-coordinator-metrics,client-id=X,name=commit-rate — Rate of offset commits

Best Practice: Always monitor lag as a trend, not just an absolute value. A lag of 100,000 is fine if it's stable; it's critical if it's growing by 10,000 messages/second.

1.3 Controller Metrics

The Kafka Controller is a special broker responsible for partition leadership elections, broker failure detection, and cluster metadata management. Controller instability can destabilize the entire cluster.

1.3.1 Controller Election Rate

MBean: kafka.controller:type=ControllerStats,name=LeaderElectionRateAndTimeMs

This metric tracks how often leadership elections occur. A non-zero rate indicates partitions are changing leaders.

- Occasional elections: Normal during rolling restarts or planned failovers
- Frequent elections: Indicates broker instability, network partitions, or GC pauses

1.3.2 Unclean Leader Elections

MBean: kafka.controller:type=ControllerStats,name=UncleanLeaderElectionsPerSec

An unclean leader election occurs when a non-ISR replica is elected as leader — meaning potential message loss. This is controlled by the unclean.leader.election.enable configuration.

WARNING: Any unclean leader election means data loss has occurred. This metric should ALWAYS be zero in production. If it is non-zero, investigate immediately and review your replication factor and min.insync.replicas settings.

1.3.3 Controller Event Queue Size

A growing controller event queue indicates the controller is overwhelmed. Check for excessive partition movements, bulk topic creations, or a struggling controller broker.

1.3.4 Controller Summary Table

Metric	Healthy Value	Warning Threshold	Action
ActiveControllerCount	1 (cluster-wide)	0 or > 1	Check ZooKeeper/KRaft, restart controller
LeaderElectionRate	0 (no elections)	Occasional spikes OK	Check for unstable brokers
UncleanLeaderElections	0 always	Any non-zero	Data loss — immediate investigation

Metric	Healthy Value	Warning Threshold	Action
EventQueueSize	< 100	> 500	Investigate controller broker load

Module 2: Monitoring Tools & Setup

Production Kafka clusters require a robust monitoring stack. The industry-standard approach combines JMX exporter, Prometheus, Grafana, and dedicated consumer lag monitoring tools like Burrow or Kminion.

2.1 JMX Monitoring

2.1.1 Enabling JMX

JMX is enabled by setting the JMX_PORT environment variable before starting the Kafka broker:

```
# Enable JMX on port 9999
export JMX_PORT=9999
./bin/kafka-server-start.sh config/server.properties
```

For remote JMX access (required for Prometheus exporter), add JVM options:

```
export KAFKA_JMX_OPTS="-Dcom.sun.jmx.remote.ssl=false \
  -Dcom.sun.jmx.remote.authenticate=false \
  -Djava.rmi.server.hostname=<broker-ip>"
```

2.1.2 Using jconsole for Ad-Hoc Monitoring

jconsole is a GUI tool bundled with the JDK for browsing JMX MBeans:

```
jconsole <broker-ip>:<JMX_PORT>
```

- Navigate to MBeans > kafka.server > ReplicaManager > UnderReplicatedPartitions
- Useful for one-off diagnostics but not suitable for production monitoring

2.2 Prometheus + JMX Exporter

The Kafka JMX Exporter is a Java agent that exposes JMX metrics as Prometheus-compatible HTTP endpoints.

2.2.1 JMX Exporter Setup

```
# Add to kafka-server-start.sh or KAFKA_OPTS
export KAFKA_OPTS="-
javaagent:/opt/jmx_exporter/jmx_prometheus_javaagent.jar=7071:/opt/jmx_exporter/kafka.
yml"
```

The kafka.yml configuration file specifies which MBeans to export. A comprehensive Kafka JMX config is available at the Prometheus GitHub repository.

```
# Verify metrics are exposed
curl http://localhost:7071/metrics | grep kafka_server_replica_manager
```

2.2.2 Key Prometheus Queries for Kafka

PromQL Query	Purpose
kafka_server_replicamanager_underreplicatedpartitions	URP count per broker
kafka_controller_kafkacontroller_activecontrollercount	Active controller check
sum(kafka_server_brokertopicmetrics_messagesinpersec) by (topic)	Messages/sec per topic
kafka_consumer_group_lag	Consumer group lag (via Kafka Exporter)
kafka_network_requestmetrics_totaltimems{quantile='0.99'}	P99 request latency

2.3 Grafana Dashboards

Grafana provides visualization for Prometheus metrics. The community maintains several excellent Kafka dashboards:

- Grafana Dashboard ID 7589 — Kafka Overview (by Confluent)
- Grafana Dashboard ID 12483 — Kafka Exporter Overview
- Grafana Dashboard ID 11962 — Kafka Consumer Group Lag

2.3.1 Essential Dashboard Panels

- Cluster Health Panel: ActiveControllerCount, OfflinePartitions, URP
- Throughput Panel: BytesInPerSec, BytesOutPerSec, MessagesInPerSec per topic
- Consumer Lag Panel: Lag trend per consumer group and topic
- Broker JVM Panel: Heap usage, GC pause duration, GC frequency
- Request Latency Panel: P50, P95, P99 Produce/Fetch latencies

2.4 Burrow — Consumer Lag Monitoring

Burrow is a dedicated open-source consumer lag monitoring tool by LinkedIn. Unlike simple offset checks, Burrow evaluates consumer health based on lag trends — distinguishing between a "working" consumer that has lag and a "stuck" consumer.

2.4.1 Burrow Status States

Status	Meaning	Action Required
OK	Consumer is making progress, lag is stable or decreasing	None
WARNING	Consumer lag is increasing but consumer is still active	Monitor closely
ERROR	Consumer is stuck — not committing offsets	Restart consumer, investigate
STOP	Consumer has stopped entirely	Immediate intervention needed
STALL	Consumer is committing but not consuming	Check consumer logic

```
# Query Burrow for consumer group status
curl http://burrow-host:8000/v3/kafka/local/consumer/my-group/status
```

Module 3: Troubleshooting Scenarios

This module covers the most common production issues encountered with Kafka, with structured diagnostic workflows and resolution steps used by L3 support engineers.

3.1 Troubleshooting Consumer Lag

3.1.1 Diagnostic Workflow

> *Symptom: Consumer group lag is growing continuously. End-to-end latency is increasing in the application.*

Step 1: Quantify the lag

```
kafka-consumer-groups.sh --bootstrap-server broker:9092 \
  --describe --group <consumer-group-id>
```

Step 2: Check if lag is on all partitions or specific ones

- All partitions lagging: Consumer throughput is insufficient for the message volume
- Specific partitions only: Consumer instance assigned to those partitions is struggling

Step 3: Check consumer throughput metrics

```
# Via JMX/Prometheus
kafka.consumer:type=consumer-fetch-manager-metrics,name=records-consumed-rate
```

Step 4: Check producer throughput — is message volume spiking?

```
kafka-consumer-groups.sh --bootstrap-server broker:9092 --describe --group <group>
# Watch LOG-END-OFFSET growing rapidly
```

3.1.2 Root Causes and Remediation

Root Cause	Evidence	Remediation
Consumer too slow	Records-consumed-rate < MessagesInPerSec	Optimize consumer logic, add parallelism
Insufficient consumer instances	Some partitions have no active consumer	Add more consumer instances
Consumer GC pauses	Consumer JVM GC logs show long pauses	Tune GC settings, reduce heap pressure
Network bottleneck	High fetch latency, consumer on slow link	Move consumer closer to Kafka, check NIC
Processing bottleneck	Consumer commits lag behind poll()	Increase processing thread pool size

Root Cause	Evidence	Remediation
Rebalancing loop	Consumer-ID changing frequently	Increase session.timeout.ms, fix consumer health

3.1.3 Consumer Configuration Tuning for Lag

Configuration	Default	Tuning Direction	Effect
fetch.min.bytes	1	Increase to 1024-65536	Fetch larger batches, fewer round-trips
fetch.max.wait.ms	500ms	Increase to 1000ms	Allow broker to accumulate more data
max.poll.records	500	Increase to 1000-5000	Process more records per poll
max.partition.fetch.bytes	1MB	Increase to 5MB-50MB	Larger partition fetch size

3.2 Troubleshooting Under-Replicated Partitions (URPs)

3.2.1 Understanding URPs

An Under-Replicated Partition exists when at least one replica in the partition's ISR (In-Sync Replica) set has fallen out of sync. This reduces your fault tolerance — if the leader fails, you may lose data or have an outage.

3.2.2 Diagnostic Workflow

Step 1: Identify which partitions are under-replicated

```
kafka-topics.sh --bootstrap-server broker:9092 --describe \
  --under-replicated-partitions
```

```
# Output example:
# Topic: orders  Partition: 3  Leader: 1  Replicas: 1,2,3  Isr: 1
# This shows broker 2 and 3 are out of ISR for partition 3
```

Step 2: Check if the affected broker is alive

```
kafka-broker-api-versions.sh --bootstrap-server broker-2:9092
# If this hangs or fails, broker-2 is down
```

Step 3: Check broker logs for replica-related errors

```
grep -i 'replica\|isr\|replication' /var/log/kafka/server.log | tail -100
```

Step 4: Check consumer lag on the broker — is it falling behind?

```
# Check replica fetch lag via JMX
kafka.server:type=ReplicaFetcherManager,name=MaxLag,clientId=Replica
```

3.2.3 Root Causes and Remediation

Root Cause	Evidence	Remediation
Broker is down	No response from broker, offline in metadata	Restart broker, check disk/OOM issues
Follower too slow	MaxLag metric high on follower	Increase replica.fetch.max.bytes, check disk
Network partition	Broker responds but cannot reach leader	Check network routes, firewall rules
Disk full	IOExceptions in broker log	Free disk space, expand storage
Leader overloaded	High request queue time on leader	Rebalance leaders, reduce producer rate
GC pause on follower	Long GC pauses in broker JVM log	Tune GC, reduce heap pressure

3.2.4 Forcing ISR Recovery

After resolving the root cause, a follower should automatically rejoin the ISR once it catches up. You can monitor this:

```
# Watch ISR recovery in real-time
watch -n 5 'kafka-topics.sh --bootstrap-server broker:9092 --describe --under-replicated-partitions'
```

Important: Never manually modify ISR — let Kafka handle it automatically. Forced ISR changes can cause data loss. The configuration replica.lag.time.max.ms (default 10s) controls how long a follower can be behind before being removed from ISR.

3.3 Troubleshooting Producer Issues

3.3.1 Producer Stuck / Not Sending Messages

> *Symptom: Application reports producer is blocked or throw TimeoutException. MessagesInPerSec on broker drops to zero for the producer's topics.*

Step 1: Check producer metrics

```
# Record send rate — should be > 0
kafka.producer:type=producer-metrics,client-id=X,name=record-send-rate
# Record error rate — check for failed sends
kafka.producer:type=producer-metrics,client-id=X,name=record-error-rate
# Request latency
kafka.producer:type=producer-metrics,client-id=X,name=request-latency-avg
```

Step 2: Check the producer log for exceptions

- TimeoutException: Producer cannot connect to broker or broker is not responding
- NotLeaderForPartitionException: Metadata is stale — broker leadership changed
- RecordTooLargeException: Message exceeds max.request.size or message.max.bytes
- BufferExhaustedException: Producer buffer (buffer.memory) is full

3.3.2 Producer Configuration Issues

Issue	Configuration	Diagnosis	Fix
Buffer full	buffer.memory (default 32MB)	BufferExhaustedException in logs	Increase buffer.memory or reduce send rate
Message too large	max.request.size, message.max.bytes	RecordTooLargeException	Increase both values consistently
Timeout connecting	request.timeout.ms	TimeoutException on connect	Check network, verify bootstrap servers
Metadata stale	metadata.max.age.ms	NotLeaderException frequently	Reduce metadata.max.age.ms

3.4 Troubleshooting Consumer Problems

3.4.1 Consumer Rebalancing Loop

> *Symptom: Consumer group is continuously rebalancing. CONSUMER-ID changes every few seconds in consumer group describe output. Application shows intermittent errors.*

Causes and fixes:

- Consumer processing takes longer than max.poll.interval.ms (default 5 minutes) — consumer is kicked out of the group
  - Fix: Increase max.poll.interval.ms or reduce max.poll.records to process less per poll
- Consumer JVM crashes due to OOM or other errors
  - Fix: Review consumer application logs, fix memory issues
- session.timeout.ms too low — heartbeats missed due to GC or load
  - Fix: Increase session.timeout.ms (must be within group.min.session.timeout.ms and group.max.session.timeout.ms broker settings)
- Consumer host networking issues causing missed heartbeats

3.4.2 Consumer Not Progressing (Committed but Not Consuming)

Stall state in Burrow — consumer commits offsets but does not actually poll for new messages. Check:

- Consumer is pausing partitions via consumer.pause() but not resuming
- Consumer is in a tight loop not calling poll() frequently enough
- Downstream system (DB, API) the consumer writes to is slow/down — consumer backs off

Module 4: Kafka Broker Log Analysis

Kafka broker logs are the single most important source of diagnostic information. A deep understanding of log patterns, error signatures, and log levels is essential for L3 support.

4.1 Log Locations and Levels

4.1.1 Log Files

Log File	Purpose	When to Check
server.log	Main broker operational log — ALL key events	Always — first stop in any investigation
controller.log	Controller-specific events — elections, topic changes	Leadership and partition issues
state-change.log	Leader/follower state transitions per partition	ISR and URP investigations
kafka-request.log	Per-request detailed logging (very verbose)	Request-level debugging only
log-cleaner.log	Log compaction operations	Topic compaction issues
kafka-authorizer.log	ACL authorization decisions	Security/access issues
kafka-gc.log	JVM garbage collection log	Memory pressure, latency spikes

4.2 Key Log Patterns to Know

4.2.1 Broker Startup and Registration

Normal healthy startup sequence in server.log:

```
[INFO] KafkaServer id=1 started (kafka.server.KafkaServer)
[INFO] Registered broker 1 at path ... (kafka.utils.ZkUtils)
[INFO] [Controller 0]: New broker startup callback for 1
(kafka.controller.KafkaController)
```

4.2.2 Leader Election Messages

```
[INFO] [Controller 0]: Elect leader for partition [orders,3]
(kafka.controller.KafkaController)
[INFO] [ReplicaFetcherManager]: Added fetcher for partitions ...
(kafka.server.ReplicaFetcherManager)
```

4.2.3 ISR Shrink/Expand

ISR changes are logged in both server.log and state-change.log:

```
# ISR Shrink — replica fell behind
[INFO] [Partition orders-3 broker=1] Shrinking ISR from 1,2,3 to 1,2 because ...
```

```
# ISR Expand — replica caught up
[INFO] [Partition orders-3 broker=1] Expanding ISR from 1,2 to 1,2,3
```

Observation: Frequent ISR shrinks on a specific broker are a strong indicator that broker is struggling. Repeated shrink/expand cycles (ISR thrashing) indicate an unstable follower.

4.2.4 Critical Error Patterns

Log Pattern	Meaning	Priority
WARN [ReplicaFetcherThread]: Error in fetch ...	Follower cannot fetch from leader	HIGH
ERROR [KafkaRequestHandler]: Error when handling request	Request processing error	HIGH
WARN [Log]: Found a corrupted index file due to ...	Log index corruption detected	CRITICAL
ERROR java.io.IOException: No space left on device	Disk full — broker will stop	CRITICAL
WARN [GroupCoordinator]: Member ... failed heartbeat	Consumer heartbeat missed	MEDIUM
INFO [KafkaController]: Broker 2 has been marked dead	Broker failure detected	HIGH
ERROR org.apache.kafka.common.errors.CorruptRecordException	Data corruption in log	CRITICAL

4.3 Log Analysis Commands

4.3.1 Searching Logs Efficiently

```
# Find all ERROR entries in last 1000 lines
tail -1000 /var/log/kafka/server.log | grep ERROR
```

```
# Find ISR changes
grep -i 'shrinking isr\|expanding isr' /var/log/kafka/server.log
```

```
# Find broker failures
grep 'marked dead\|shut down\|shutdown' /var/log/kafka/server.log
```

```
# Check controller elections
grep 'ResigningAsLeader\|becomes active controller\|Elect leader'
/var/log/kafka/controller.log
```

```
# Check for disk issues
grep -i 'IOException\|No space left\|disk' /var/log/kafka/server.log
```

```
# Check GC impact
grep -i 'GC overhead\|OutOfMemoryError\|java.lang.OOM' /var/log/kafka/server.log
```

4.3.2 Correlating Timestamps

When correlating events across broker logs, use timestamps precisely:

```
# Find all events within a time window
awk '/2024-01-15 14:30/,/2024-01-15 14:35/' /var/log/kafka/server.log
```

```
# Count errors per minute
grep 'ERROR' /var/log/kafka/server.log | awk '{print $1, $2}' | cut -c1-16 | sort |
uniq -c
```

Module 5: Simulating Consumer Lag

Controlled simulation of consumer lag is an important skill for understanding Kafka behavior, testing monitoring alerting, and training incident response. This module covers how to intentionally induce lag and observe the system's behavior.

5.1 Methods to Simulate Consumer Lag

5.1.1 Method 1: Slow Consumer

The simplest method — introduce artificial delay in consumer processing:

```
# Python example — slow consumer
from confluent_kafka import Consumer
import time

c = Consumer({'bootstrap.servers': 'localhost:9092',
              'group.id': 'lag-test-group',
              'auto.offset.reset': 'earliest'})
c.subscribe(['test-topic'])

while True:
    msg = c.poll(1.0)
    if msg:
        time.sleep(0.1)  # 100ms delay per message = max 10 msgs/sec
        # But do NOT commit — lag will grow
```

5.1.2 Method 2: High-Volume Producer

Send messages faster than the consumer can process:

```
# Use kafka-producer-perf-test.sh to flood a topic
kafka-producer-perf-test.sh \
  --topic test-topic \
  --num-records 1000000 \
  --record-size 1024 \
  --throughput -1 \
  --producer-props bootstrap.servers=localhost:9092
```

5.1.3 Method 3: Stop Consumer, Keep Producing

Stop the consumer group entirely while production continues:

```
# 1. Start producing (in background)
kafka-console-producer.sh --topic test-topic --bootstrap-server localhost:9092 &
```

```
# 2. Stop all consumer instances for the group
# (Kill the consumer processes)

# 3. Watch lag grow
watch -n 2 'kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --describe --group test-group | grep -v CONSUMER'
```

5.2 Observing and Resolving Induced Lag

5.2.1 Monitoring During Simulation

- Watch LAG column increasing in kafka-consumer-groups.sh output
- Observe records-lag-max metric climbing in Prometheus/Grafana
- Check Burrow status transitioning from OK → WARNING → ERROR

5.2.2 Recovery Strategies

1. Reset consumer offset to latest (skip lagging messages — data loss risk):

```
kafka-consumer-groups.sh --bootstrap-server broker:9092 \
  --group test-group --topic test-topic \
  --reset-offsets --to-latest --execute
```

2. Scale up consumers (add more consumer instances up to partition count):

```
# Start additional consumer instances in the same group
# They will receive reassigned partitions via rebalancing
```

3. Optimize consumer processing to increase throughput
4. Temporarily pause non-critical consumers to reduce broker load

5.3 Lag Simulation Lab Summary

Scenario	Expected Behavior	Recovery Action
Stop consumer, keep producing	Lag grows continuously per partition	Start consumer, let it catch up or reset to latest
Slow consumer (100ms delay)	Lag grows if producer rate > 10 msgs/sec	Optimize consumer, reduce sleep, add parallel consumers
Producer throughput spike	Lag grows temporarily then stabilizes	Usually self-resolving if consumer rate is adequate
Consumer rebalancing loop	Lag grows during each rebalance cycle	Fix consumer stability, tune timeouts

Day 3 Summary & Key Takeaways

Critical Metrics Checklist

Metric	Healthy Value	Alert Threshold
ActiveControllerCount	Exactly 1 (cluster-wide)	0 or > 1
UnderReplicatedPartitions	0	> 0 for > 30 seconds
OfflinePartitionsCount	0	Any value > 0
Consumer Lag	0 or stable, low value	Continuously growing lag
UncleanLeaderElections	0	Any non-zero value
RequestHandlerIdleRatio	> 0.5 (50%+)	< 0.2 (< 20%)
LogFlushRateAndTimeMs	< 500ms	> 1000ms sustained

Troubleshooting Quick Reference

- Consumer Lag Growing: Check consumer throughput, add consumers, optimize processing
- URPs Present: Check broker health, disk space, network between brokers
- No Active Controller: Check ZooKeeper/KRaft connectivity, restart controller broker
- Producer Stuck: Check buffer.memory, broker connectivity, topic authorization
- Rebalancing Loop: Increase max.poll.interval.ms, fix consumer stability

Log Analysis Quick Reference

- Always start with server.log — it contains the complete operational timeline
- ISR changes → state-change.log for partition-level detail
- Controller events → controller.log for election and metadata issues
- GC impact → kafka-gc.log correlated with request latency spikes